

Using Sequential and Divide-And-Conquer Methods to Identify The Most Popular Dock

Fatmanur Bulut-230401034

Samet Ünsal-220401082

Department of Computer Engineering
İzmir Katip Çelebi University

November 22, 2025

Abstract

This report compares two algorithms to find the most active dock. The data is stored in a binary matrix where rows represent the docks and columns represent time slots. A value of 1 means the dock is busy. The goal is to find the dock with the highest usage count.

We looked at a simple method that reads the data row by row, and a 'divide-and-conquer' method that splits the data into parts to count. The report analyzes the speed and memory usage of both methods and discusses the test results. The results show that the simple method is easier to code, but the divide-and-conquer method can be better when there are a lot of time slots.

1 Introduction

Distribution centers use loading docks to handle goods coming in and going out. Managers need to know how well these docks are being used. Just looking at a list of arrival times isn't enough. They need a simple way to track activity and compare the docks fairly.

Raw data logs can be confusing because some docks have many short visits, while others have a few long ones. To solve this, we convert the data into a "binary matrix." This basically divides the day into equal time slots. If a dock is busy during a slot, we mark it as 1; if it's empty, it's 0. This gives us a standard way to compare time.

Once we have this data, the main goal is to find which dock is used the most. In this report, the "most utilized" dock is simply the one with the highest number of busy slots. This simple number helps us see if a dock is overworked or if it is sitting empty and wasting space.

We explore two algorithms to solve this. The first is a "sequential" method. It goes through the matrix row by row and counts the ones. It is very straightforward. The second is a "divide-and-conquer" method. This approach splits the matrix into smaller parts to handle the data differently. Both methods look at all the entries, but they manage memory and structure in different ways.

The rest of the report is organized as follows: Section 2 defines the problem and shows how we build the matrix. Section 3 explains the two algorithms. Section 4 analyzes their speed and complexity. Section 5 presents our test results, and Section 6 concludes the report with ideas for future work.

2 Problem Definition and Data Preparation

2.1 Binary Model of Dock Usage

In this project, we define R as the number of docks and T as the number of time slots in a day. We use a matrix U to track when docks are busy. The matrix has size $R \times T$ and contains only zeros and ones:

$$U \in \{0, 1\}^{R \times T}.$$

If dock i is busy during time slot t , we set $U[i, t] = 1$. If it is free, we set it to 0. The total usage count for a specific dock i is the sum of all the ones in its row:

$$C[i] = \sum_{t=1}^T U[i, t].$$

Our main task is to find the dock i^* that has the highest count $C[i^*]$. If there is a tie (two or more docks have the same maximum number), we simply pick the one with the smallest index number:

$$i^* = \min\{i \mid C[i] = \max_j C[j]\}.$$

Both algorithms we discuss later use this same rule to make sure they give the exact same answer.

2.2 Choosing the Time Slots

To build the matrix U , we have to split the continuous time of a day into fixed chunks. For this study, we decided on a slot length of $\Delta = 5$ minutes. Since there are 1440 minutes in a full day, the total number of columns in our matrix is:

$$T = \frac{1440}{5} = 288.$$

We chose 5 minutes because it is a good balance. If the slots were smaller, the matrix would be too big. If they were larger, we might miss short visits.

Each time slot represents a time interval $[t_{start}, t_{end})$. This means the start time is included, but the end time is not. This rule helps us avoid counting the same event twice if it ends exactly when a new slot begins.

2.3 Handling the Raw Data

Before we can analyze anything, we take the raw data logs, which look like this:

$$(dock_id, t_{in}, t_{out}).$$

Here, $dock_id$ tells us which dock it is, and t_{in} and t_{out} are the start and end times. We convert these times into minutes from the start of the day. Then, we figure out which column (time slot) they belong to.

We use a simple rule: if a visit overlaps with a time slot at all, we mark that slot as 1. This ensures we don't ignore short visits that happen inside a 5-minute window.

We assign a row number (from 1 to R) to each dock. Also, if we find any bad data—like a visit that ends before it starts—we just ignore it to keep the results clean.

2.4 Data Summary

Once the matrix U is ready, we look at a few key numbers to understand the dataset:

- The number of docks, R .
- The number of time slots, T .
- The total number of busy slots (1s) in the whole matrix:

$$N_{\text{ones}} = \sum_{i=1}^R \sum_{t=1}^T U[i, t].$$

- The sparsity, which tells us how "empty" the matrix is:

$$\text{sparsity} = 1 - \frac{N_{\text{ones}}}{R \cdot T}.$$

If the sparsity is high, it means most docks are empty for most of the day. If it is low, the distribution center is very busy. These numbers help us explain the test results later on.

3 Algorithms

In this section, we describe the two algorithms we used to identify the dock with the highest activity: the sequential scan and the divide-and-conquer method.

3.1 Sequential Algorithm

The sequential algorithm is the most straightforward approach. It basically does exactly what the problem asks for. It goes through the matrix row by row, counts the ones in each row, and remembers the best row found so far. If two docks end up with the same count, the algorithm picks the one with the smaller index number, as required by the tie-breaking rule.

3.1.1 Pseudo-Code

```
best_row    = 1
best_count  = 0

for i in 1..R:
    count = 0
    for t in 1..T:
        count = count + U[i,t]
    if count > best_count:
        best_row    = i
        best_count  = count
    else if count == best_count and i < best_row:
        best_row = i

return (best_row, best_count)
```

3.1.2 Discussion and Correctness

This method looks at every single cell in the matrix exactly once. For every dock i , the variable `count` holds the correct total sum $C[i]$ after the inner loop finishes. The outer loop makes sure that `best_row` always stores the winner among the rows we have checked so far. It also handles the tie-breaking rule correctly. So, when the loops are done, `best_row` is definitely the answer we are looking for (i^*).

The sequential method is very simple to code in any language. It is also a good baseline to compare other algorithms against.

3.2 Divide-and-Conquer Algorithm

The divide-and-conquer algorithm works differently. It uses the idea that the total count for a dock is just the sum of counts from different parts of the day. Instead of scanning the whole row at once, it splits the matrix into two halves (left and right), counts them separately, and then adds the results together.

3.2.1 Pseudo-Code

```
function row_counts(U):
    # U has shape R x T
    if T == 1:
        # Base case: only one column left
        return U[:,1]

    # Split by columns into left and right halves
    split column range into [1..T_left] and [T_left+1..T]
    UL = U[:, 1..T_left]
    UR = U[:, T_left+1..T]

    CL = row_counts(UL)
    CR = row_counts(UR)
```

```

# Combine partial results
C = CL + CR # element-wise addition
return C

function best_row_from_counts(C):
    best_row = 1
    best_count = C[1]
    for i in 2..R:
        if C[i] > best_count:
            best_row = i
            best_count = C[i]
        else if C[i] == best_count and i < best_row:
            best_row = i
    return (best_row, best_count)

# Main D&C algorithm
C = row_counts(U)
(best_row, best_count) = best_row_from_counts(C)
return (best_row, best_count)

```

3.2.2 Discussion and Correctness

The logic starts with the base case: if there is only one column ($T = 1$), the function just returns that column because no addition is needed. In every other step, the matrix is cut in half. The recursive calls give us the partial counts for the left side ($C_L[i]$) and the right side ($C_R[i]$). Then, we combine them:

$$C[i] = C_L[i] + C_R[i].$$

Since we are just adding the parts together, this mathematically guarantees that we get the correct total for row i . After getting the totals, the second function `best_row_from_counts` finds the maximum value and handles the tie-breaking, just like the sequential algorithm. This ensures the final result is exactly $(i^*, C[i^*])$.

The main benefit of this approach is that it breaks the work into smaller blocks. In real-world applications, this structure can be better for computer memory (cache) and makes it easier to run the code on multiple processors at the same time.

4 Complexity Analysis

4.1 Time Complexity

Analyzing the time complexity for the sequential algorithm is quite simple. The code uses a nested loop structure: it goes through R rows, and for each row, it checks T columns. Since the work done inside the loop is constant ($O(1)$), the total time is:

$$T_{\text{seq}}(R, T) = O(RT).$$

For the divide-and-conquer algorithm, it's a bit different. The function `row_counts` keeps splitting the matrix in half. At every step of the recursion, it has to iterate through all R rows to combine the results from the left and right sides. If we call the running time $T_{\text{dc}}(R, T)$, we can write the recurrence relation as:

$$T_{\text{dc}}(R, T) = 2T_{\text{dc}}(R, T/2) + O(R)$$

for $T > 1$. The base case is $T_{\text{dc}}(R, 1) = O(R)$. By applying the Master Theorem, the solution to this recurrence is:

$$T_{\text{dc}}(R, T) = O(RT).$$

So, mathematically speaking, both algorithms take linear time relative to the size of the matrix.

4.2 Memory Complexity

The sequential algorithm is very efficient with memory. Apart from the matrix itself, it only needs a tiny amount of extra space for variables like the loop counter, the current maximum, and the best row index. This means the auxiliary space complexity is $O(1)$.

The divide-and-conquer algorithm requires more memory. It needs space for the recursion stack and for storing the partial count vectors (C_L and C_R) at each step. If we assume the matrix isn't copied but referenced by indices, the main cost is the recursion. The depth of the recursion tree is $O(\log T)$, and at each level, we store a vector of size R . Therefore, the extra memory usage is about $O(R \log T)$.

4.3 Practical Considerations

Even though both methods have the same "Big-O" time complexity, they behave differently in the real world. The sequential algorithm reads the matrix row by row. This is usually very fast because it accesses memory in a contiguous way, which computers like. The divide-and-conquer method does the same amount of work but has the overhead of function calls.

For small to medium datasets, the overhead of recursion usually makes the divide-and-conquer method slower than the simple sequential loop. However, if the matrix is huge, the divide-and-conquer approach might catch up. This is because smaller submatrices might fit better in the CPU cache, or because the independent parts can be calculated in parallel on different processors.

5 Experimental Evaluation

In this section, we describe how we tested the two algorithms and how they performed. The main goal wasn't to get perfect timing numbers, but to compare them and see how they handle larger datasets.

5.1 Experimental Setup

We wrote the code in Python using the NumPy library to handle the matrices. The tests ran on a standard laptop with 16 GB of RAM. To make the comparison fair, we used the exact same input matrix U for every test run.

To see how the algorithms react to different sizes, we created several versions of the matrix by:

- changing the number of docks R (like 20, 50, or 100),
- changing the number of time slots T (making the day longer or shorter),
- changing the "density" (making the docks busier or quieter).

For every test case, we ran the algorithms multiple times and calculated the average time they took to finish.

5.2 Results and Discussion

First of all, both the sequential and the divide-and-conquer algorithms gave the exact same results ($i^*, C[i^*]$). This confirms that our code works correctly and handles the tie-breaking rule properly.

For small matrices (for example, when $R \leq 50$ and $T \leq 300$), the sequential algorithm was usually a bit faster. This makes sense because the divide-and-conquer method has extra work to do—like setting up recursion and managing data structures—which isn't worth it for small tasks.

However, as we increased T (adding more time slots), the time needed for both algorithms went up linearly, as expected. But for these larger datasets, the divide-and-conquer method started to catch up. Breaking the matrix into smaller chunks seemed to help with memory (cache) efficiency, allowing Python to process the data more smoothly.

The experiments showed that there is no "best" algorithm for every situation. The sequential method is better for simple tasks because it's easier to write and has less overhead. The divide-and-conquer method becomes a better choice when the matrix is very wide or if we want to run the code on multiple processors at once.

6 Conclusion

In this report, we presented a complete solution to the “Most Utilized Dock” problem. We compared two main strategies: a simple sequential scan and a divide-and-conquer approach. We modeled the dock activity using a binary matrix created from event logs, and our goal was to identify the dock with the highest number of busy time slots, using a specific rule to break ties.

We described both algorithms in detail and explained why they are correct. Our complexity analysis showed that both methods theoretically run in $\mathcal{O}(RT)$ time, where R is the number of docks and T is the number of time slots. However, they differ in memory usage: the sequential algorithm uses very little extra memory (constant space), while the divide-and-conquer algorithm needs more space ($\mathcal{O}(R \log T)$) because of the recursion and temporary storage.

Our experiments showed that the sequential method is generally faster for small and medium-sized datasets because it is simple and has very low overhead. On the other hand, the divide-and-conquer algorithm showed potential for very large matrices, especially because its structure helps with cache efficiency and sets the stage for parallel processing.

7 Bibliography

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. McKinney, W. (2017). *Python for Data Analysis* (2nd ed.). O’Reilly Media.
3. OpenAI *ChatGPT* (2025). Used as writing assistants for language refinement and paraphrasing.
4. Google *Gemini* (2025). Used as writing assistants for language refinement and paraphrasing.